Quick Sort. (Devide and Concare.

eg:-

35 50 15 25 80 20 90 45.

↓ ↓p

Pivot element. Q Q Q.

→ p shd be greater than equal to pivot value.

→ Q shd be less than equal to pivot value.

→ P. value is 50 greater than 35, yes, stop move ahead.

→ Next come to Q, Q is 45, 45 less than 35, No, so move Q ahead.

→ Now 90, less than 35, no, so move ahead

→ 20, 20 less than 35, yes, so Q is stopped here, P is already stopped.

→ Both P & Q is stoppped, and

It can be swapped.

Lesser than pivot

→ 35, 20, 15, 25, 80, 50, 90, 45 +∞.

P check greater than of pivot value. P. Q. Q.

But if they came equally or crossed, then you have to do it with the pivot.

<u>Now</u>

25 20 15 $\boxed{35}$ 80 50 90 45.

pivot value is placed in right position

i.e

35

check greater

v 25, 20, 15 +∞   80, 50, 90, 45 +∞

P Q   v   P Q.

We can used Divide & conceire.

Now check P greater than pivot value.

till it reached +∞.

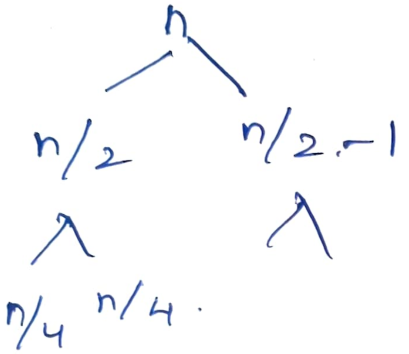
Now swap P & Q.

80, 50, 45, 90 +∞

v P Q.

$$T(n) = T(n/2) + T(n/2) + n.$$

$$= 2T(n/2) + n$$ → scan entire array



→ If full array is completed then it is 1 pass completed.

⇒ $\boxed{N \log N}$ → Average time complexity of Quick Sort.

7. bipartite graph Matching.
8. N-Queen problem. ADC.
9. Job sequence with deadline problem.

1. Knapsack problem.
2. Backtracking approch using N-Queen Problem.
3. Job Sequencing using Greedy.
4. Dijkstra.
5. Graph coloring problem.
6. Multistage Graph / Dynamic ~~programming~~ programming Approch for finding shortest path in a Multigraph.

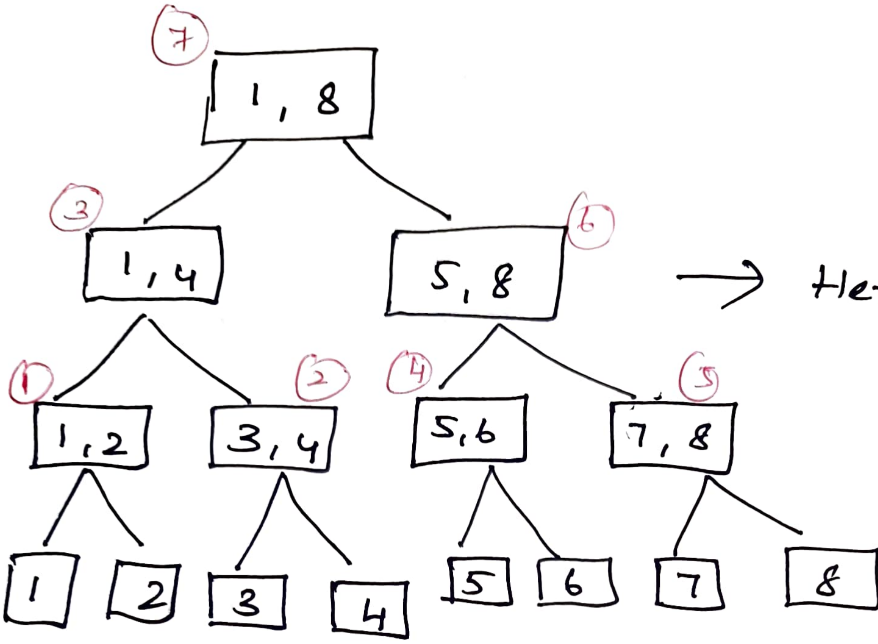
Merge Sort.

→ It follow Divide & conco

hence it is called as Recursion.

```
Algorithm Merge sort. (l,h)
{
 if (l<h)
 {
  mid = (l+h)/2;
then perform Merge Sort (l, Mid);
       Merge sort (mid +1, h);
       Merge (l, mid, h);
 }}.
```

	1	2	3	4	5	6	7	8
A	9	3	7	5	6	4	8	2
	l			↑ mid				h.

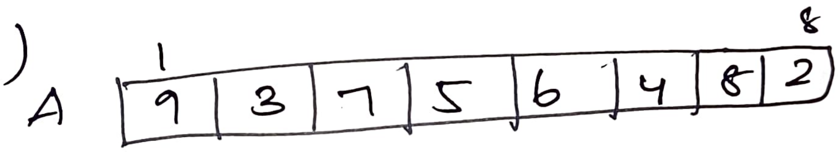


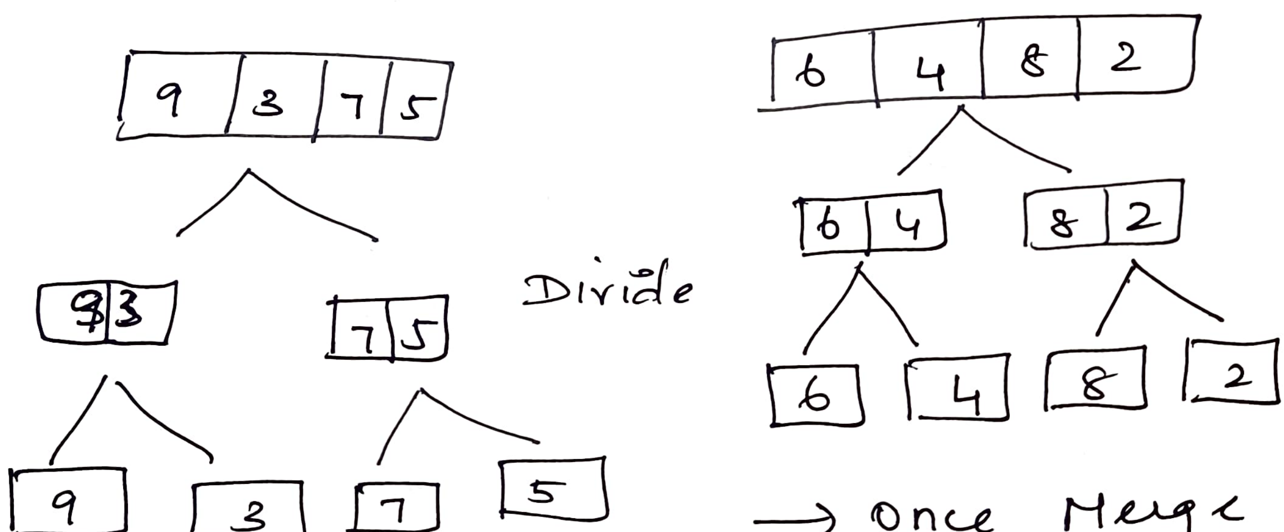
→ Here its Divided.

First order
↳ left
Merge is post order.

For 8 element we have 3 levels.

O(nlogn)

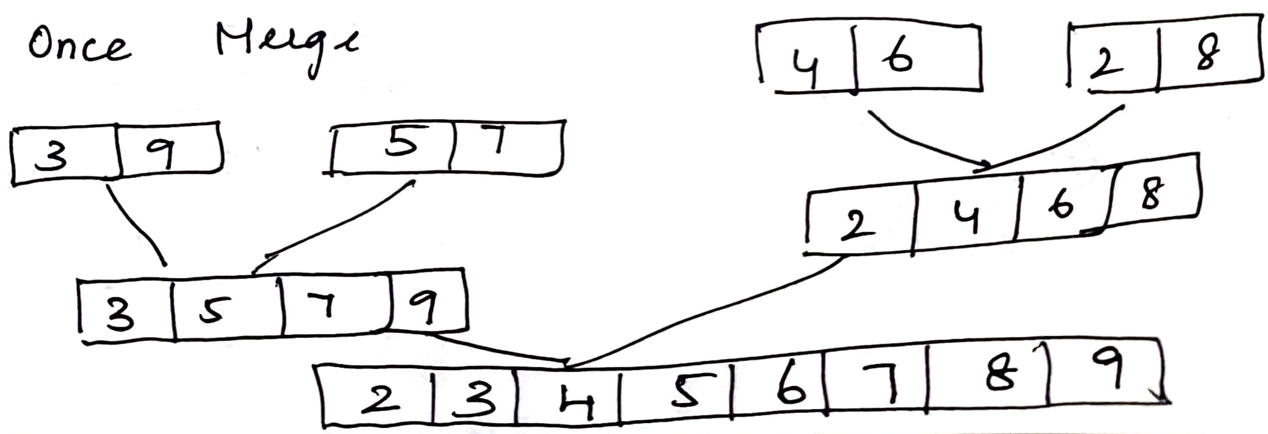




Divide

→ Once Merge

→ Once Merge



Minimum Spanning Tree.

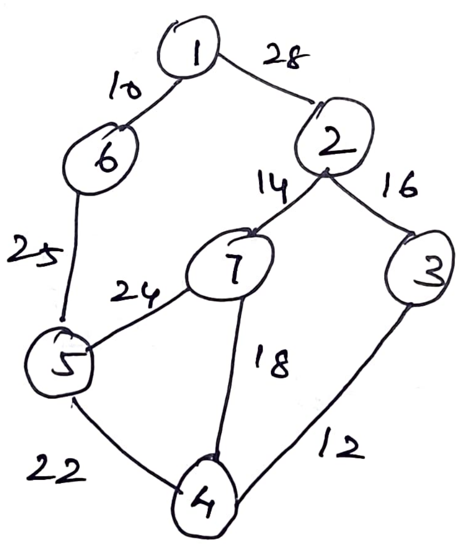
Greedy Methods.

↳ If weighted graph is given then it can be found out.

↳ prim

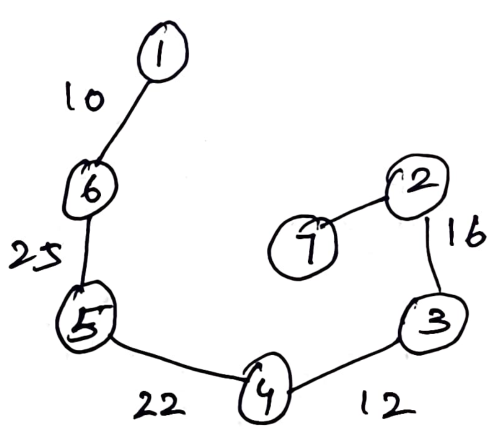
↳ krushal.

Prim's Algorithm.



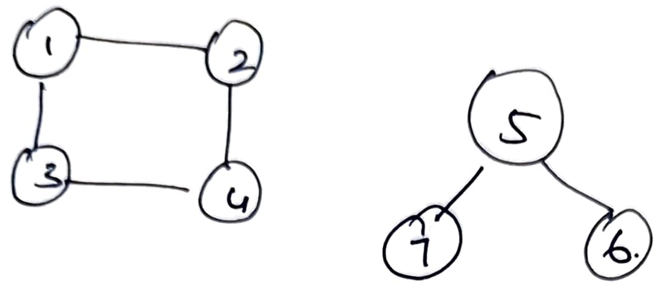
V = 7

E = 6



initially select small & select the node which is small.

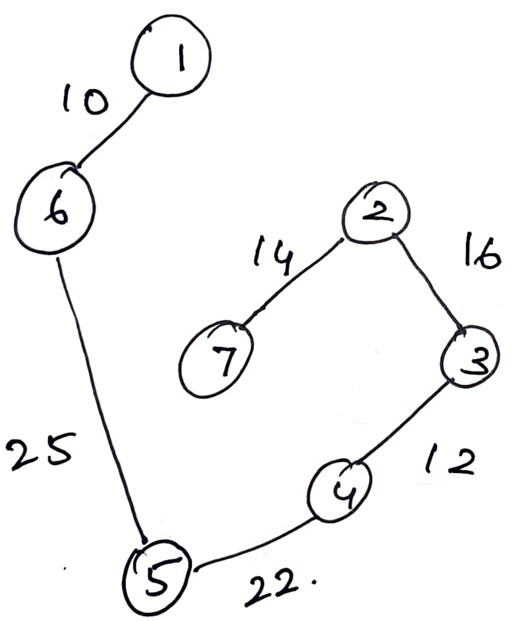
⇒ cost = 99.



For non-connected component it cannot be found.

2. Kruskal's

Ans:



Cost = 99

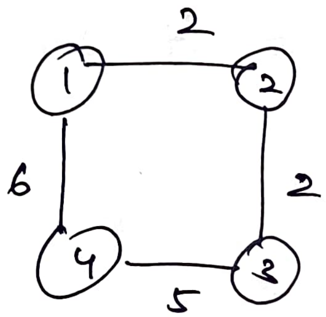
$$|E| = |V| - 1$$
$$= 7 - 1$$
$$= 6$$

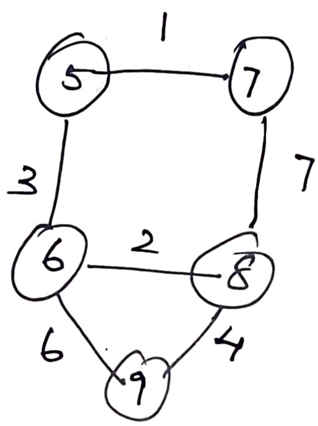
$$\Theta(|V|\,|E|)$$
$$\Theta(n \cdot e)$$
$$\Theta(7 \cdot 6)$$

Time taken by kruskal $\Big\} = \Theta(n^2)$

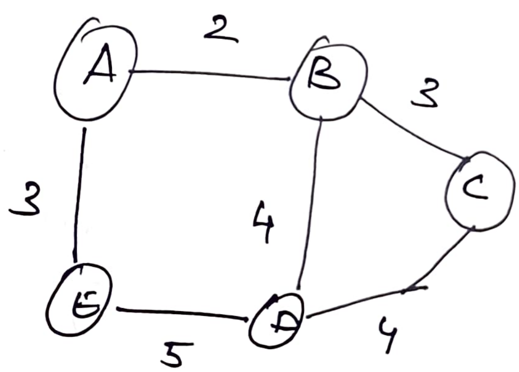
min Heap ⇒ Time Complexity.

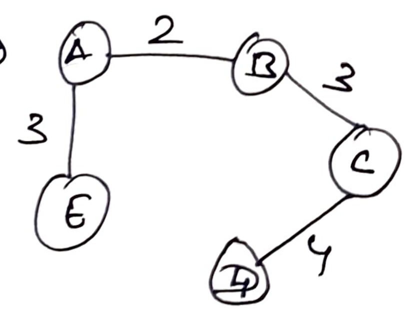
$\Theta(n \log n)$





eg :





$\Rightarrow 2+3+3+4 = 12$

$\Rightarrow$ Ans

12.

Huffman Coding

Reduce the size of data/file.
It is like a zip file.
Fixed size / variable size

Example:

Message → BCCABBDDAECCBBAE
DDCC

Length → 20.
ASCII codes – 8 bits.

	→ ASCII code for A.	→ Binary 8 bit represe
A	65	01000001
B	66	
C	67	
D	68	
E	69.	

character	count / frequency	code.
A	3/20	000
B	5/20	001
C	6/20	010
D	4/20	011
E	2/20	100

Always chart / code with message shd be sent.

Table size = 5 Alaphabet × 8 bits.

⇒ 5×8

⇒ 40. bits.

5 Alap × 3 bit (own)

⇒ 5×3

= 15

⇒ 40+15 ⇒ 55 bits.

Msg/Charater size = 20 message × 8 bits (ascii)

Now we can write our own code

= 20 message × 3 bits

= 20×3

= 60 bits.

Msg = 60 bits.

Table = 55 bits.

115 bits.

20×8 bits

⇒ 160 bits

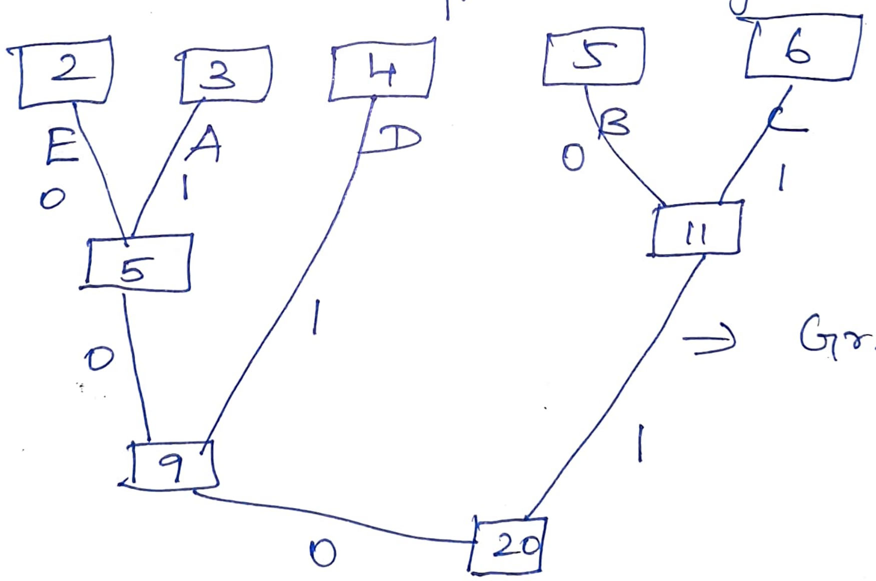
By using Huffman coding

Char	Count	Code	count ↓ × code no ↓
A	3	001	$3 \times 3 = 9$
B	5	10	$5 \times 2 = 10$
C	6	11	$6 \times 2 = 12$
D	4	01	$4 \times 2 = 8$
E	2	000	$2 \times 3 = 6$
$5 \times 8 \Rightarrow 40$ bits	20	12 bits.	45 bits

↳ New code obtained

+

→ Arrange in ascending



→ Greedy Approach

Left side → Mark as '0'

Right side → Mark as '1'

Now encode the Message with the New code obtained.

Message Size = 45 bits.

Along with the msg sent the table/chart.

Preserve Tree/Table.

5 Alpabets $\Rightarrow 5 \times 8$

$= 40$ bits.

msg $= 45$ bits.

Tree/Table $= 40 + 12$

$= 52$ bits.

Total $=$ msg $+$ table

$= 52 + 45$

$= 97$ bits.

From Tree we can get the size.

$\sum d_i * f_i$

$d_i \leftarrow$ Distance &

frequency $\hookrightarrow f_i$

$\Rightarrow 3\times2 + 3\times3 + 2\times4$

$+ 2\times5 + 2\times6.$

$\Rightarrow 45$ bits.

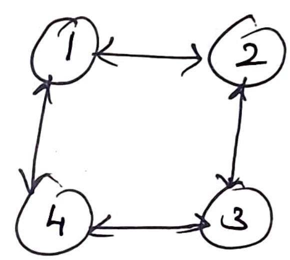
Traveling Sales man Problem - Dynamic Programm

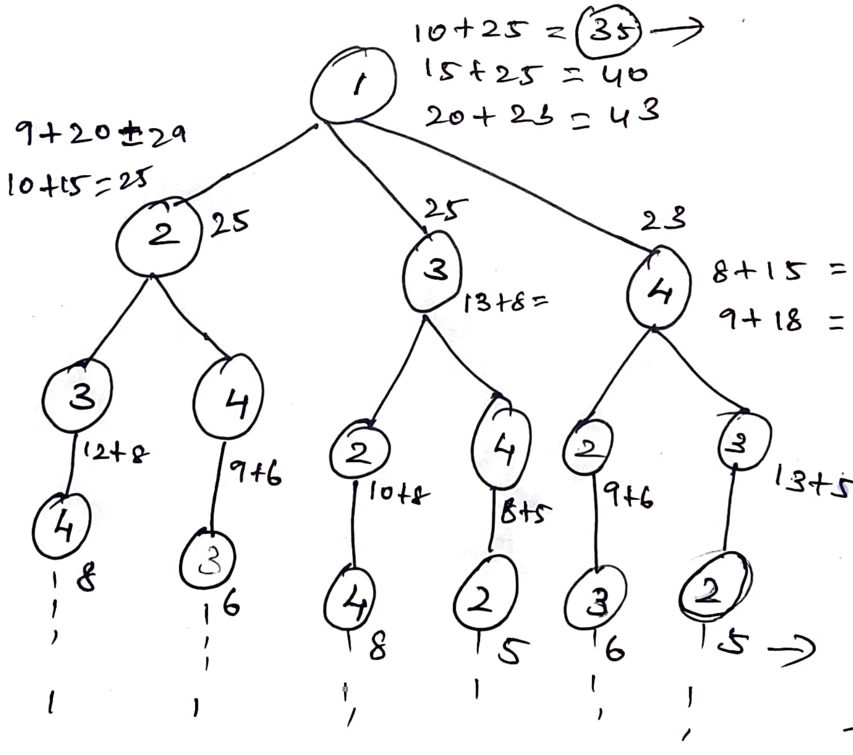
$g(4,3) = 15$
$g(2,4) = 18$
$g(3,2) = 18$
$g(3,4) = 20$
$g(4,2) = 13$

Try
and find
is the best.

$$
A = \begin{array}{c|cccc}
 & 1 & 2 & 3 & 4 \\
\hline
1 & 0 & 10 & 15 & 20 \\
2 & 5 & 0 & 9 & 10 \\
3 & 6 & 13 & 0 & 12 \\
4 & 8 & 8 & 9 & 0
\end{array}
$$

Brute- force.





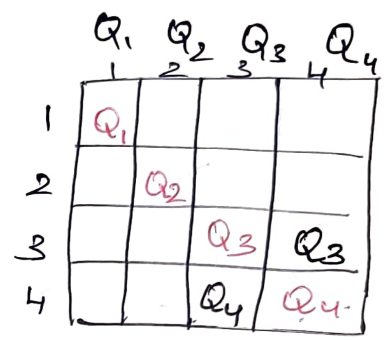
N-Queens.

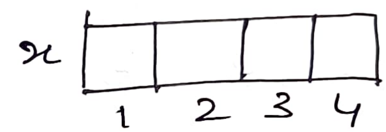
Same → row/colum/ diagonal then it will get attacked.

$^{16}C_4$ → ways

Avoid keeping 2 queen in same place.

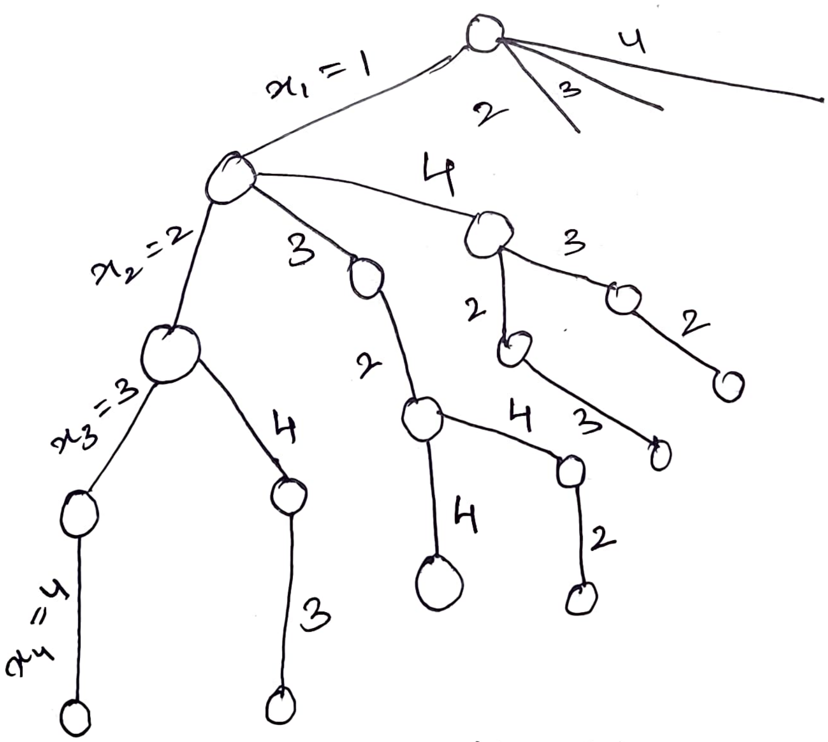
4-Queens





Column No.

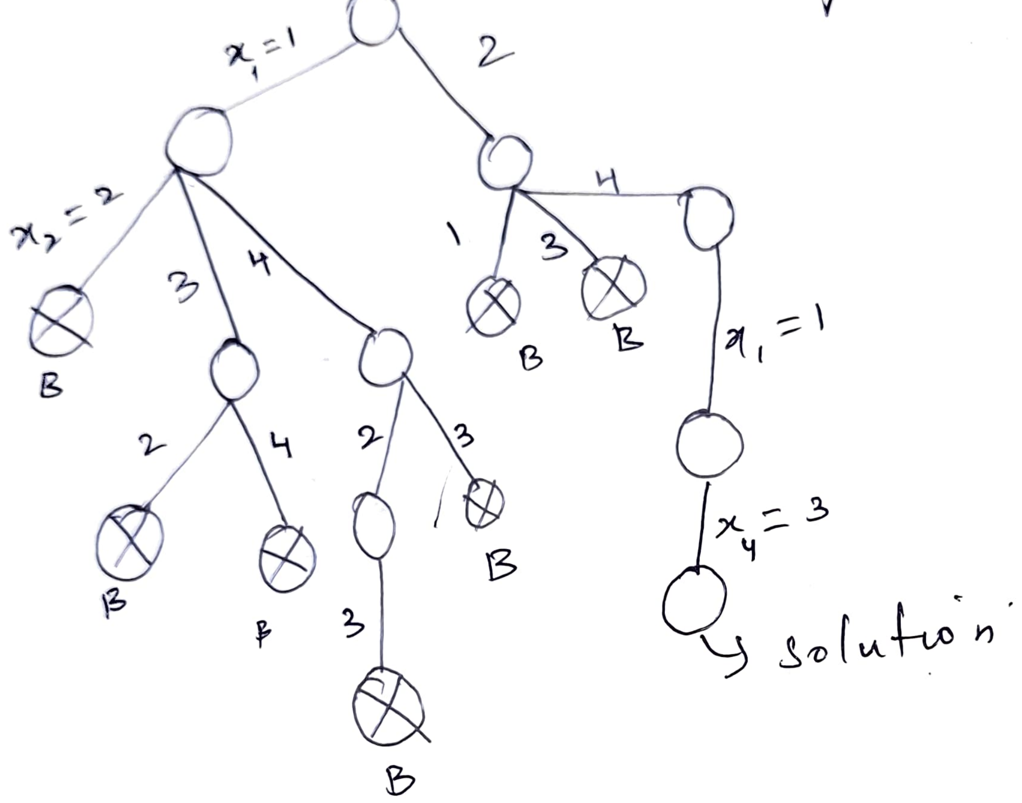
State - Space Tree.



$1+4+4\times3+4\times3\times2+4\times3\times2\times1$

$\Rightarrow 1+\sum_{i=0}^{3}\left[\prod_{j=0}^{i}(4-j)\right] \Rightarrow$ 65 nodes.

Bounding function - using Back tracking.



Solution : | 2 | 4 | 1 | 3 | x.

Another answer : Mirror image.

| 3 | 1 | 4 | 2 |

1. Find optimal solution using Greedy for the job sequencing with deadline problem with the following values.

$N = 5$

Profit = $\{10, 3, 33, 11, 40\}$

Deadline = $\{3, 1, 1, 2, 2\}$.

Step 1: Pair Jobs with Their Profit and deadlines.

Job 1: profit = 10, Deadline = 3

Job 2: Profit = 3, Deadline = 1

Job 3: profit = 33, Deadline = 1

Job 4: Profit = 11, Deadline = 2

Job 5: Profit = 40, Deadline = 2

Step 2: Sort Jobs by profit in Descending Order.

Job 5 : profit = 40, Deadline = 2

Job 3 : Profit = 33, Deadline = 1

Job 4 : Profit = 11, Deadline = 2

Job 1 : profit = 10, Deadline = 3

Job 2 : Profit = 3, Deadline = 1

Step 3: Create a schedule array to keep track of which slots are occupied up to the max deadline.

Since the highest deadline is 3, we need an array of size 3 (i.e. schedule = [-1, -1, -1], where each index represents a time slot.

Step 4: Schedule the Jobs.

We attempt to schedule each job in the latest available slot before its deadline.

1. Job 5 (profit = 40, Deadline = 2): The latest available slot before or at deadline 2 is slot 1.

   Schedule Job 5 in slot 1

   schedule = [-1, Job 5, -1]

2. Job 3 (Profit = 33, Deadline = 1): The latest available slot before or at deadline it is slot 0.

   Schedule Job 3 in slot 0.

   Schedule = [Job 3, Job 5, -1]

3. Job 4 (profit = 11, Deadline = 2): The latest available slot before or at deadline 2 is slot 2.

Schedule = [Job 3, Job 5, Job 4]

4. Job 1 (profit = 10, Deadline = 3): All slots up to deadline 3 are occupied, so skip Job 1.

5. Job 2 (profit = 3, Deadline = 1): All slots up to deadline 1 are occupied, so skip Job 2.

Step 5: calculate the optimal solution

The scheduled Jobs are Job 3, Job 5, Job 4, with profits of 33, 40 and 11, respectively.

Total profit = 33 + 40 + 11 = 84.

Step 6:

optimal solution set of jobs to max profit within their deadline is.

Job 3, Job 5, and Job 4 with a total profit of 84.

1. Apply Greedy Mtd to obtain Optimal Solution to the Knapsack problem with

$M = 60$

$W = \{5, 10, 20, 30, 40\}$

$P = \{30, 20, 100, 90, 160\}$

Find total profit earned.

Step 1: Calculate profit-to weight Ratios (frac {P} {W}):

Item 1: (frac {30} {5})

$= 6.$

Item 2: (frac {20} {10})

$= 2$

Item 3: (frac {100} {20}) = 5

Item 4: (frac {90} {30}) = 3

Item 5: (frac {160} {40}) = 4.

Step 2: Sort items by profit-to weight. Ration (Descending)

Item 1 = 6
Item 3 = 5
Item 5 = 4
Item 4 = 3
Item 2 = 2.

} from step 1.

Step 3: Add items to the knapsack

1. Item 1: $(w = 5)$, $(P = 30)$.
   $(M = 60)$

   $\therefore$ Remaining capacity $= (60 - 5 = 55)$
   Total profit $= (30)$

2. Item 3 : $(w = 20)$ $(P = 100)$ $(M = 55)$
   from step 3.

   $\therefore$ Remaining capacity $= (55 - 20) = 35$
   Total profit $= [30 + 100 = 130]$.

3. Item 5 : $W = 40$, $P = 160$, $M = 35$
   Remaining capacity

Item 5 exceeds the remaining capacity of 35, so take fraction of it

Fraction taken = (frac {35} {40}

= 0.875)

Profit from fraction = (0.875 times

160 = 140)

Total profit = (130 + 140 = 270)

The optimal Solution with the Greedy methods yields a total profit of 270.